

Lab4 :

Diabetic Retinopathy Detection

311552055 蕭育泓

1. Introduction

這次作業主要的內容是先把視網膜圖片 resize 成 512*512，並且每個圖片都有可能對應到 5 個不同的 label，然後利用 ResNet 來建立捲積神經網路，而 ResNet 的特色是在每一層中使用 skip connection，使得神經網路可以避免，因為太多層而導致梯度消失或爆炸等問題，而這次作業主要是利用 ResNet18 與 ResNet50 分別利用 pre-train 和 no pre-train model 來實作。

2. Experiment setups

A. The details of your model (ResNet)

```
def initialize_model(model_name, num_classes, feature_extract, use_pretrained=True):  
    """ Resnet50  
    """  
    model_ft = models.resnet50(pretrained=use_pretrained)  
    num_ftrs = model_ft.fc.in_features  
    model_ft.fc = nn.Linear(num_ftrs, num_classes)  
  
    return model_ft
```

```
def initialize_model(model_name, num_classes, feature_extract, use_pretrained=True):  
    if model_name == "Resnet18":  
        """ Resnet18  
        """  
        model_ft = models.resnet18(pretrained=use_pretrained)  
  
        num_ftrs = model_ft.fc.in_features  
        model_ft.fc = nn.Linear(num_ftrs, num_classes)
```

使用 torchvision 中的套件 models 來實作 ResNet18、ResNet50，而在 function initialize_model 中最後面的 parameter(use_pretrained)如果是 true 就會是使用 pre-train 模型，反之就是不使用。

B. The details of your Dataloader

使用自定義的 dataloader，而在 def __init__ 中，我們將使用到 function getData 來將我們 csv 檔中，每個圖片的名字和對應 label 名字給取出

來，接著我們會利用 `def __getitem__` 把取出來的圖片名所對應的照片給 load 出來，並且將他轉成 tensor 型態後配合 label 一起 return。

```
def __init__(self, root, mode, augmentation=None):
    """
    Args:
        root (string): Root path of the dataset.
        mode : Indicate procedure status(training or testing)

        self.img_name (string list): String list that store all
        self.label (int or float list): Numerical list that sto
    """

    self.root = root
    self.img_name, self.label = getData(mode)
    self.mode = mode
    print("> Found %d images..." % (len(self.img_name)))
```

```
def getData(mode):
    if mode == 'train':
        img = pd.read_csv('train_img.csv')
        label = pd.read_csv('train_label.csv')
        return np.squeeze(img.values), np.squeeze(label.values)
    else:
        img = pd.read_csv('test_img.csv')
        label = pd.read_csv('test_label.csv')
        return np.squeeze(img.values), np.squeeze(label.values)
```

```
path = os.path.join(self.root, self.img_name[index] + '.jpeg')
img = Image.open(path)
to_tensor = transforms.ToTensor()
img = to_tensor(img)

label = self.label[index]
return img, label
```

```
test_dataset = dataloader.RetinopathyLoader("./data/new_test", "test")
train_dataset = dataloader.RetinopathyLoader('./data/new_train', 'train')
```

C. Describing your evaluation through the confusion matrix

在 test model 的時候，把 confusion 所需要的資訊先記錄下來，利用 `def plot_confusion_matrix` 來輸出成 confusion matrix。

而觀察 confusion matrix 後發現，在沒有 pre-train model 的情況下，我們的 confusion matrix 只會預測 0，導致 confusion matrix 顯示很不平衡，而有 pre-train model 的情況下，confusion matrix 顯示得就比較成對角線。

```
confusion_matrix = np.zeros((num_class, num_class))

for i in range(len(label)):
    confusion_matrix[int(label[i])][int(predict_class[i])] += 1

confusion_matrix = confusion_matrix / \
    confusion_matrix.sum(axis=1).reshape(num_class, 1)

def plot_confusion_matrix(confusion_matrix):
    fig, ax = plt.subplots(figsize=(6, 6))
    ax.matshow(confusion_matrix, cmap=plt.cm.Blues)
    ax.xaxis.set_label_position('top')
    for i in range(confusion_matrix.shape[0]):
        for j in range(confusion_matrix.shape[1]):
            ax.text(i, j, '{:.2f}'.format(
                confusion_matrix[j, i]), va='center', ha='center')
    ax.set_xlabel('Predicted label')
    ax.set_ylabel('True label')
    return fig
```

3. Data Preprocessing

A. How you preprocessed your data?

```
def get_path(mode):
    if mode == 'train':
        img = pd.read_csv('train_img.csv')
        # label = pd.read_csv('train_label.csv')
        img_name = np.squeeze(img.values)
        # label = np.squeeze(label.values)
        for index in range(len(img_name)):
            path = os.path.join("../data/new_train", img_name[index] + '.jpeg')
            image(path)
    else:
        img = pd.read_csv('test_img.csv')
        img_name = np.squeeze(img.values)
        for index in range(len(img_name)):
            path = os.path.join("../data/new_test", img_name[index] + '.jpeg')
            image(path)
    return path
```

```

def image(path):
    print(path)
    img = read_image(path)

    # img_tensor = transforms.ToTensor()(img)
    C, H, W = img.size()
    # 中心裁剪为 224x224 大小的图像
    if H == W:
        return
    if H < W:
        center_cropped = transforms.CenterCrop((H, H))(img)
    if H > W:
        center_cropped = transforms.CenterCrop((W, W))(img)
    # 将裁剪后的 Tensor 转换为 PIL Image，并保存
    center_cropped_img = transforms.ToPILImage()(center_cropped)

    transform = transforms.Compose([
        transforms.Resize((512, 512)), # 缩放到 256x256 大小
        transforms.ToTensor(), # 转换为 Tensor
    ])

    img_transformed = transform(center_cropped_img)

    # 输出变换后的 Tensor 大小
    # print(img_transformed.size()) # 输出 torch.Size([3, 256, 256])
    center_cropped_img = transforms.ToPILImage()(img_transformed)
    # center_cropped_img.show()
    center_cropped_img.save(path)

```

```

if __name__ == '__main__':
    # read a JPEG image
    get_path("train")
    get_path("test")
    # image("./data//new_test//3798_left.jpeg")

```

首先，利用 `def get_path` 來把圖片的路徑抓出來，接著利用 `def image` 來把圖片 `read` 進來成為 `tensor` 的型態，然後把圖片以中心先裁減成長寬等長的圖片，再把他轉回 `PIL` 的型態，接著再把他 `resize` 成 `512*512` 的大小，最後存檔以便之後使用。

B. What makes your method special?

先把資料去除掉一些不必要的黑邊，讓每個圖片的大小都一致，然後再

生成 model 時，就可以省去 resize 的時間，也就是先把圖片做資料前處理。而我因為不希望在每次 load data 時都要做處理資料集的問題，所以把這部分單獨寫成一個.py，並且可以直接看我的圖片是否有裁切完整。

4. Experimental results

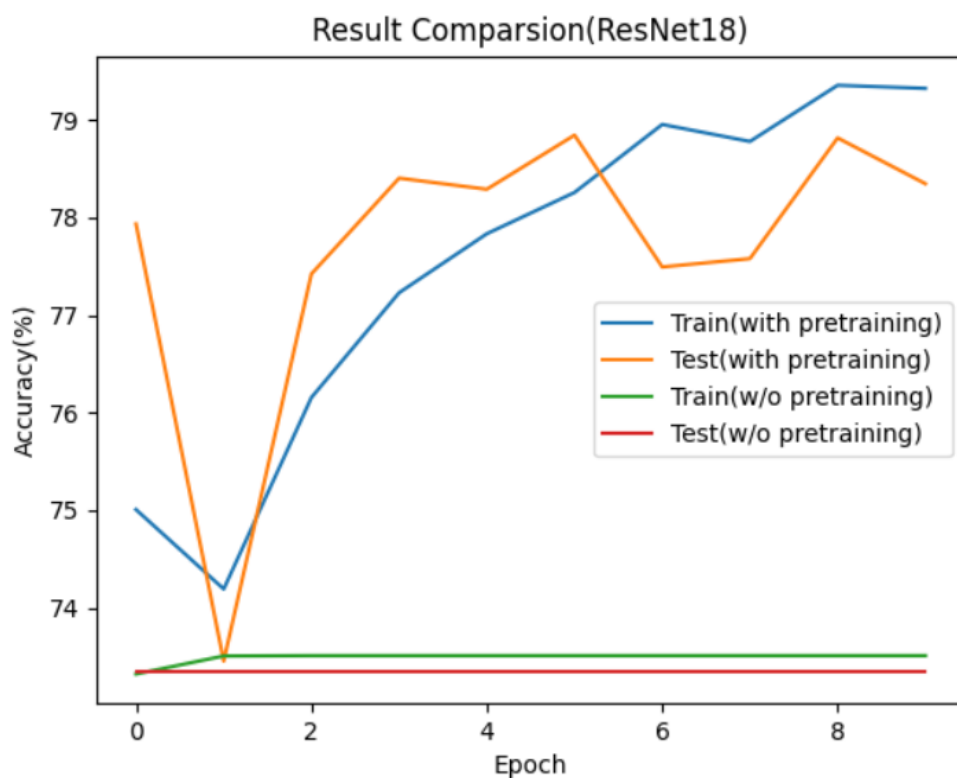
A. The highest testing accuracy

```
epoch 4 Accuracy: 76.67%, loss: 0.9692  
Test: Accuracy: 77.94%, loss: 0.6928
```

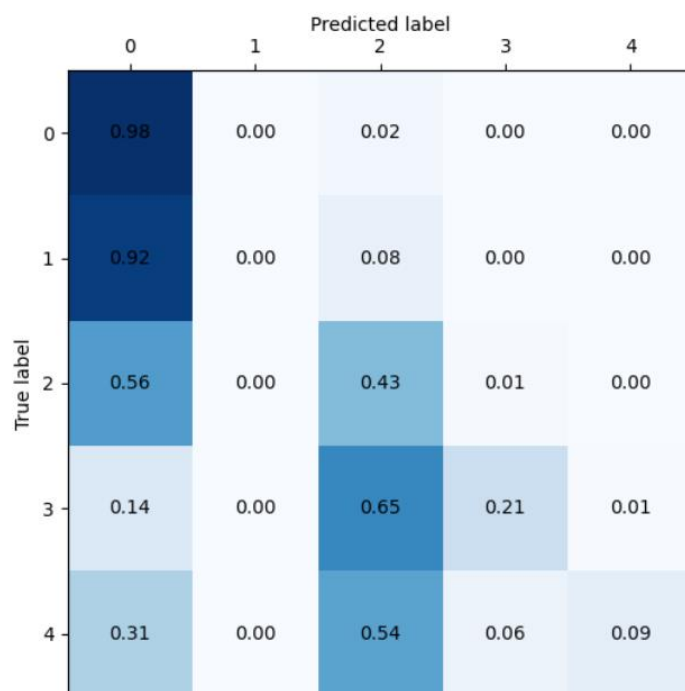
B. Comparison figures

ResNet18 :

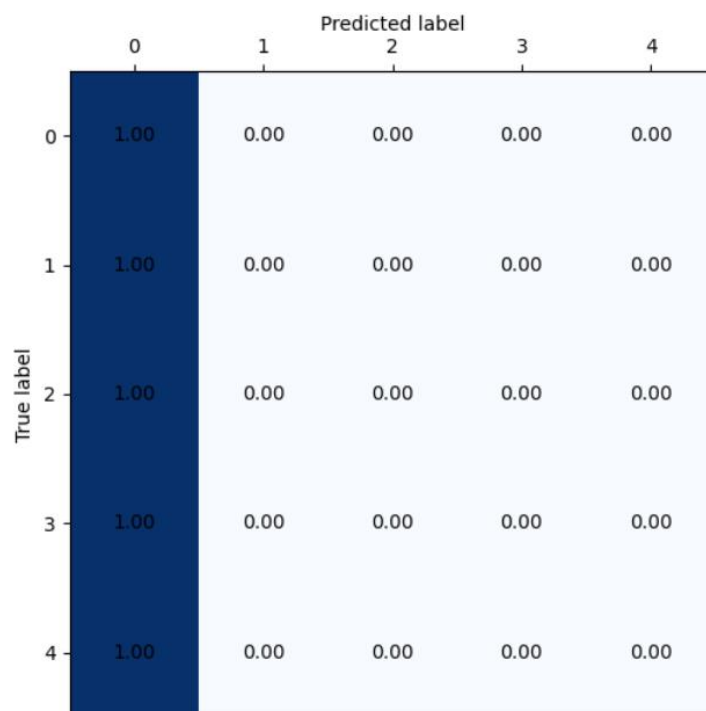
Comparison figures



With pretraining

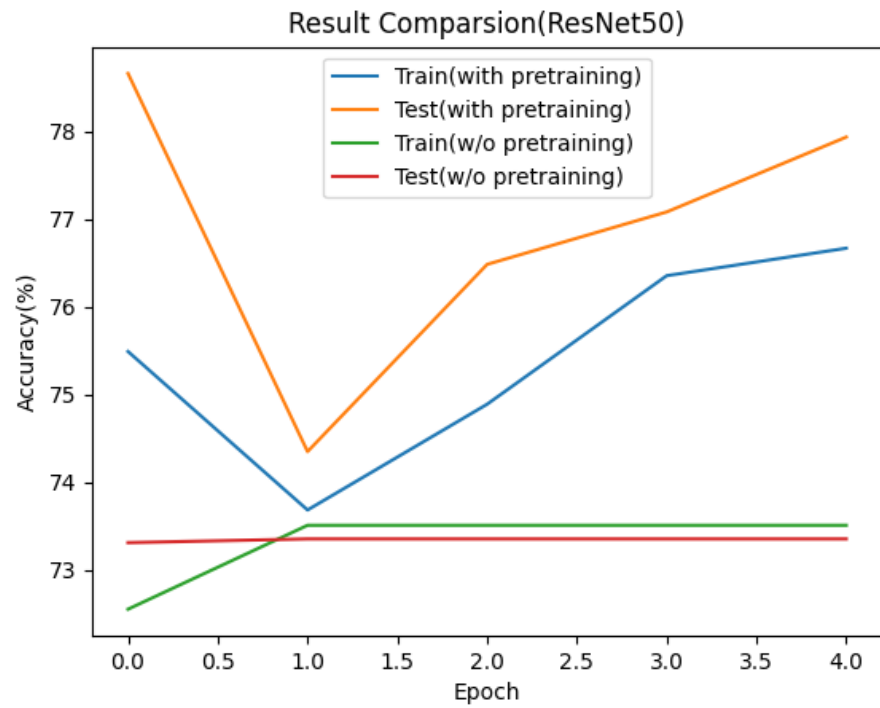


Without pretraining

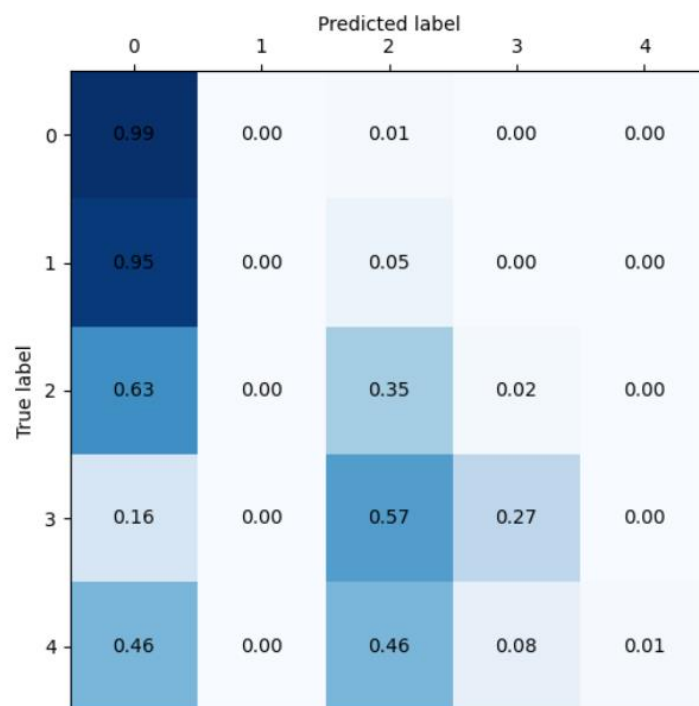


ResNet50 :

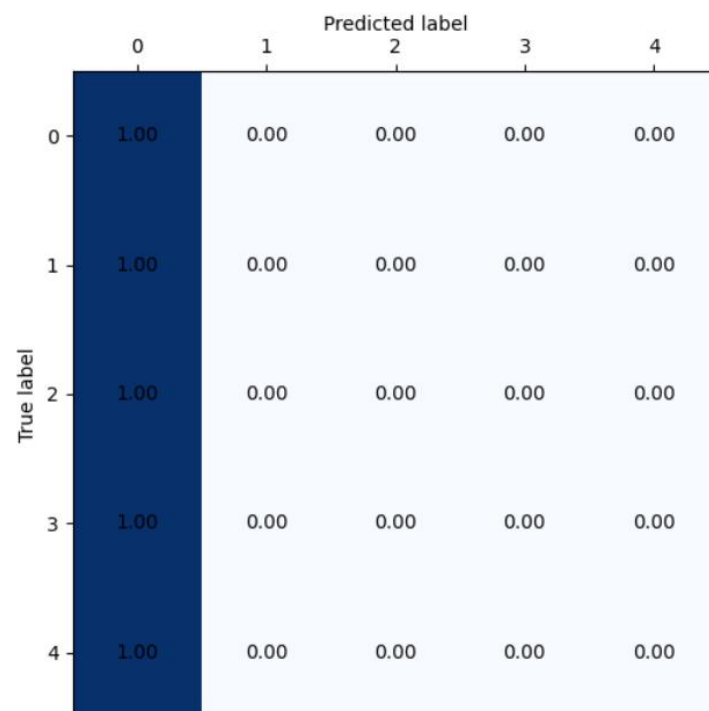
Comparison figures



With pretraining



Without pretraining



5. Discussion

1. 在做這次實驗時，在一開始用學校給的 1060 上跑的時候，圖片不知道什麼原因導入不了，導致我最終使用家裡的 3060ti 來做，但又發現原本打算用 batch size = 64 來用，去導致 cuda 已滿，所以最終使用 batch size = 4。
2. 還發現了幾個比較有趣的點是在，原本利用 ppt 給的 reference 做 feature extracting，但是後來利用下面函式後，發現效果沒有比較好，

```
def set_parameter_requires_grad(model, feature_extracting):  
    if feature_extracting:  
        for param in model.parameters():  
            param.requires_grad = False
```

所以就直接所幸不用了，所以只訓練了最後一層的權重，表示做 fin-tuning。

3. 有做 pre-train 的效果都比沒有的還要來的好，沒做的都在預測時，全部預測為沒有問題，這可能也跟資料集中，大多數的眼球是被判斷為 label 0 有關。